

M E S P I W O R D F
B A C L O C K D A N
E A N A L L E E F T
M A N U A L N J L K

Anleitung

Für die Multilayout-ESP-Wordclock

Inhaltsverzeichnis

1	Einleitung	1
1.1	Hauptfunktionen	1
1.2	Uhren Varianten	2
1.3	Voraussetzungen	2
2	Installation der Wortuhr Software	3
2.1	Erstmaliges Aufspielen mittels Binary	3
2.2	Windows	3
2.3	MacOS	4
2.4	Linux	4
2.5	Update der Firmware über das Web-Frontend	4
3	Hardware	5
3.1	Pegelanpassung	6
3.2	Anschlussbeispiel	6
3.3	Konfigurierbare Hardware-Pins	7
3.3.1	Standard-Pins	8
3.3.2	Geeignete Pins je Controller	8
3.3.3	Bedienung der Hardware-Taster	10
4	Funktionen der Uhrensoftware auf dem ESP	11
4.1	Verbinden der Uhr mit dem eigenen WLAN	11
4.2	Zeitserver (NTP Server)	11
4.3	Automatische Helligkeitsregelung	11

1 Einleitung

Diese Anleitung soll helfen, eine mehrsprachige Wortuhr zu bauen, die auf einem ESP8266 Mikrocontroller und einer programmierbaren LED-Leiste (WS2812 oder SK6812) basiert. Dieses Projekt wurde über mehrere Jahre von einer Vielzahl von Mitwirkenden auf Github entwickelt und frei zur Verfügung gestellt.

Eine Wortuhr (auch als Wordclock bekannt) ist ein schönes DIY-Projekt für Anfänger, das Technologie und Design kombiniert, um eine funktionelle und ästhetisch ansprechende Uhr zu schaffen. Egal, ob Sie Anfänger oder erfahrener Bastler sind, dieses Projekt ist eine großartige Gelegenheit, Ihre Fähigkeiten unter Beweis zu stellen und etwas wirklich Besonderes zu schaffen. Die Gestaltungsmöglichkeiten liegen vor allem im Rahmen und in der Wahl der Oberfläche der Uhr.

1.1 Hauptfunktionen

Die bereitgestellte Software zum Betreiben der Wortuhr hat mehrere Funktionen:

- Mehrsprachig (Deutsch, Englisch, Niederländisch, Italienisch, Spanisch, Ungarisch, Französisch, Römänisch, Schweizer Deutsch, Russisch und Schwedisch)
- Unterstützung für mehrere Matrizengrößen und LED-Abstände
- Farbwechsel der Anzeigefarbe möglich (RGB oder RGBW)
- Digitale Uhranzeige
- Regenbogenfarbwechsel
- Umgebungslicht (als Sekundenzeiger ausgeführt oder Ambientlight)
- Automatische Helligkeitsregelung (optional über Sensor)
- Offlinebetrieb (optional über entsprechende RTC)

- Auswahl an dialektspezifischen Anzeigen
- Home-Assistant-Einbindung mit Autodiscovery

1.2 Uhren Varianten

1.3 Voraussetzungen

Die folgende Hardware/Software wird für dieses Projekt benötigt:

Hardware

- NodeMCU oder vergleichbares Board mit einem dem ESP8266 oder ESP8285 Chip
- WS2812B RGB-LED-Streifen oder SK6812 RGBW-Streifen
- Stromversorgung 5V 2A
- Optional: LDR, 10 KOhm-Widerstand
- Optional: BH1750 (digitaler Helligkeitssensor, anstatt LDR)
- Optional: DS3231 (Echtzeituhr, für den offline Betrieb)

Software

- PlatformIO Core oder IDE (bspw. Visual Studio Code)
- Node.js
- Git

2 Installation der Wortuhr Software

2.1 Erstmaliges Aufspielen mittels Binary

Für das erstmalige Aufspielen liegen auf der Githubseite Rechts «Releases» zur Verfügung. Dabei handelt es sich um fertig compilierte Versionen der Software. Programme zum aufspielen der binary findet man z.B. unter «<https://github.com/nodemcu/nodemcu-flasher>»

2.2 Windows

1. Installieren Sie die aktuellsten Versionen von PlatformIO, Node.js und Git manuell
2. Dadurch wird Visual Studio Code installiert, mit einem PlatformIO-Symbol (Ameisenkopf/Alien) in der Seitenleiste
3. Gehen Sie zu «Quick Access / Miscellaneous» und geben Sie den Befehl «Clone Git Project» ein, und geben Sie <https://github.com/ESPWortuhr/Multilayout-ESP-Wordclock> als URL ein
4. Gehen Sie dann zu «Projekte», fügen Sie das neue Projekt mit «Vorhandenes hinzufügen» zur Liste hinzu und klicken Sie auf «Öffnen».
5. In der PlatformIO-Seitenleiste erscheint nun «Project Tasks». Wählen Sie den Befehl «General / Upload'» (dauert ein paar Minuten, die Software wird zuerst erstellt).
6. Schließen Sie den ESP8266 über USB an. Wenn die Wortuhr-Software erstellt ist, wird sie auf dem ESP installiert.

2.3 MacOS

Der einfachste Weg wäre mit «homebrew»:

```
1    brew install platformio
2    brew install node
3    git clone https://github.com/ESPWortuhr/Wortuhr
4    cd Wortuhr
5    pio run -t upload
```

Dabei ist der ESP8266 über USB angeschlossen. Die Wortuhr-Software wird dabei mittels Plattformio erstellt und auf den ESP übertragen.

2.4 Linux

```
1    python3 -c "$(curl -fsSL https://raw.githubusercontent.com/...
      platformio/platformio/master/scripts/get-platformio.py) "
2    sudo apt install npm
3    git clone https://github.com/ESPWortuhr/Wortuhr
4    cd Wortuhr
5    pio run -t upload
```

Dabei ist der ESP8266 über USB angeschlossen. Die Wortuhr-Software wird dabei mittels Plattformio erstellt und auf den ESP übertragen.

2.5 Update der Firmware über das Web-Frontend

Mit der Software ist es möglich, die Firmware **OTA** (over the air), über den in der Uhr eingebauten Webserver zu aktualisieren. Dies ist in den meisten Fällen einfacher als das direkte Flashen. Ausschlaggebend für diese Vorgehensweise ist, dass die Uhr über WLAN erreichbar ist. Die Update-Schnittstelle ist dann über die Adresse `http://<IP-Adresse der Uhr>:81/update` erreichbar (Beispiel: `http://192.168.4.1:81/update`). Die Adresse der Uhr wird in der Grundeinstellung beim Start als Laufschrift ausgegeben.

3 Hardware

Für den Aufbau der Wortuhr wird folgende elektronische Hardware benötigt:

- NodeMCU oder vergleichbares Board mit ESP8266 oder ESP8285 Chip
- RGB-LED-Streifen WS2812B oder RGBW-Streifen SK6812
- Stromversorgung 5V 2A
- Optional: Hardware-Taster für Power, Modus und Geschwindigkeit/Farbe
- Optional: LDR, 10 KOhm Widerstand
- Optional: BH1750 (digitaler Helligkeitssensor, ersetzt LDR)
- Optional: DS3231 (Echtzeituhr, für Offline-Betrieb)

Ich möchte dich gerne darüber informieren, was du noch alles für den Bau benötigst. Da es verschiedene Arten gibt, eine Uhr zu bauen, werde ich dir hier nur die grundlegenden Dinge aufzählen, die ich oft verwende:

- Bilderrahmen (Holz oder Metall mit mindestens 14mm Rahmenhöhe)
- Diffusorfolie
- Buchstabenmaske
- Abstandsgitter (Holz oder Kunststoffschaum)

3.1 Pegelanpassung

Der ESP arbeitet intern mit einer Betriebsspannung von 3,3V. Die WS2812(B) Streifen arbeiten dagegen mit 5V. Dies erfordert eine Pegelanpassung, damit die digitalen Signale fehlerfrei zu den Streifen übertragen werden können. Ohne Anpassung kann es auch funktionieren, muss es aber nicht. Es gilt: "Der minimale High-Pegel des Ausgangs muss höher sein als der minimale High-Pegel des Eingangs" Was heißt das im Klartext? Laut Datenblatt des Streifen sind folgende Pegel notwendig, um ein sauberes HIGH und LOW Signal zu bekommen. Um ein HIGH Signal am Eingang des WS2812B Streifen zu erzeugen, muss laut Datenblatt (Bild 1) mindestens ein Pegel von 4,3V erzeugt werden. Da der ESP aber maximal 3,3V erzeugen kann, wird hier kein sicheres HIGH Signal erzeugt.

Nun kommt der Trick mit der Diode: Die erste LED wird nun mit einer Diode in der Plus Leitung betrieben. Die Durchlassspannung der Diode beträgt 0,7V. Das heißt, dass die erste Diode mit 4,3V betrieben wird. Das ist laut Datenblatt auch zulässig. Bei 4,3V beträgt der benötigte HIGH Pegel nun noch 3,6V. Da die Hersteller noch Toleranzen mit einberechnen funktioniert das ganze Konstrukt wieder. Nun gibt die erste LED den Pegel mit 4,3V an die nächste LED weiter (diese wird wieder mit 5V betrieben und der HIGH Pegel liegt mit 4,3V wieder im Datenblatt in der Toleranz). Die zweite LED verstärkt sozusagen das Signal wieder auf die volle 5V. Das heißt, die erste LED wird sozusagen als Pegelanpassung missbraucht.

3.2 Anschlussbeispiel

Der Datenpin des LED Streifen ist in der Grundeinstellung an dem Port «Rx» (GPIO3) angeschlossen. Damit bleibt die bisherige ESP8266-Ansteuerung mit festem RX-Pin kompatibel. Wird der LED-Pin in der Hardware-Konfiguration geändert, verwendet die Firmware auf dem ESP8266 automatisch die Bit-Bang-Ansteuerung für den gewählten GPIO. Wie auf dem Schaltbild muss eine Diode in die +5V Leitung der 1. LED. Die +5V Leitung zwischen der 1. und 2. LED muss durchtrennt werden (aber nur diese, nicht die Daten und GND!). Am 2. Streifen wird dann zusätzlich vom Netzteil die +5V eingespeist. Die LED's vom 1. Streifen werden dann sozusagen rückwärts mit den nötigen +5V versorgt.

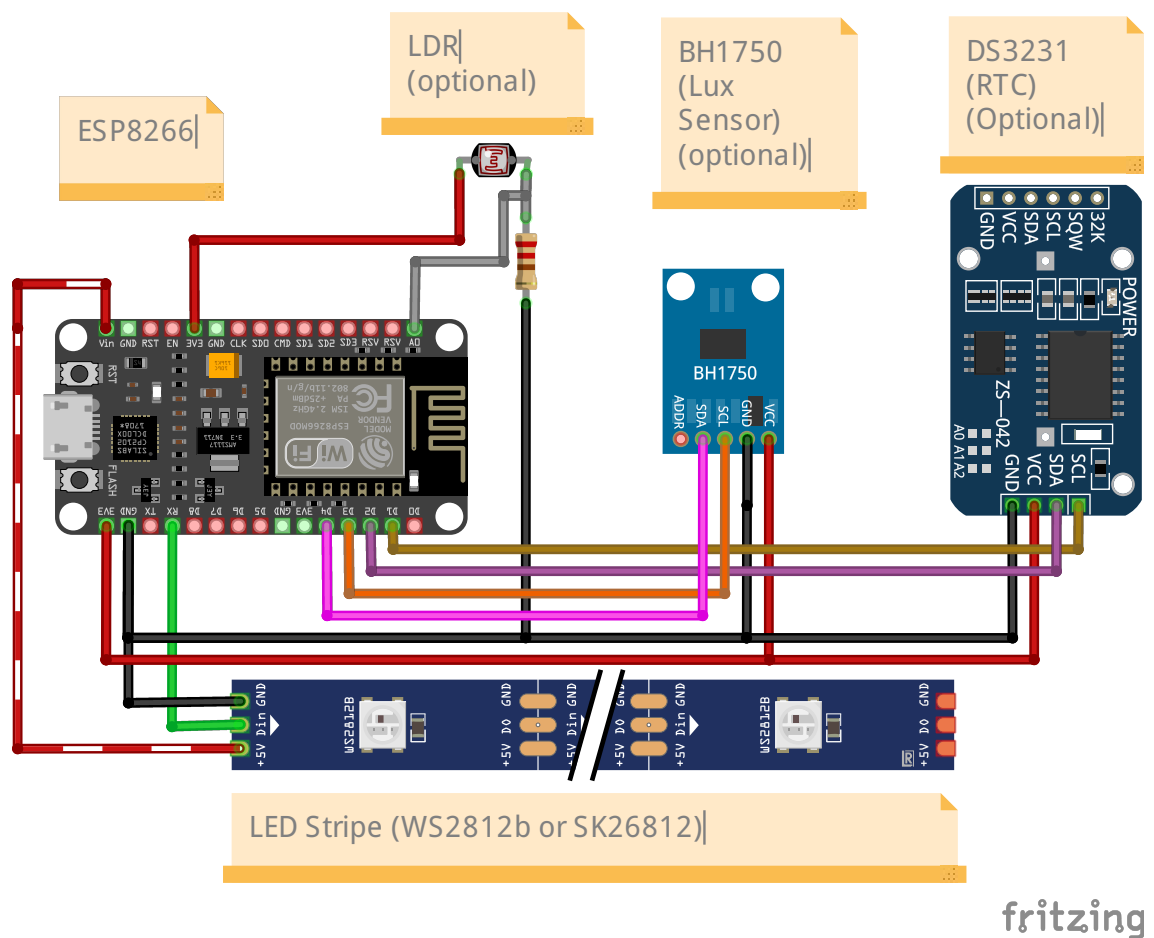


Abbildung 3.1: Anschlussbeispiel

3.3 Konfigurierbare Hardware-Pins

Die Firmware kann den Datenpin des LED-Streifens und drei optionale Hardware-Taster konfigurieren. Die Konfiguration wird im Web-Frontend unter «Einstellungen» im Bereich «Hardware-Pins» vorgenommen. Es muss immer die GPIO-Nummer des Controllers eingetragen werden, nicht die auf dem Board aufgedruckte Bezeichnung. Bei NodeMCU-Boards ist zum Beispiel «D4» nicht die richtige Eingabe, sondern die zugehörige GPIO-Nummer.

Hardware-Pins

Hier können die GPIO-Nummern für den LED-Streifen und die Hardware-Tasten eingestellt werden. Es muss die GPIO-Nummer verwendet werden, nicht die aufgedruckte Board-Bezeichnung.

LED-Streifen

Power-Taste

Mode-Taste

Speed-Taste

Abbildung 3.2: Hardware-Pin-Konfiguration im Web-Frontend

3.3.1 Standard-Pins

Funktion	Standard-GPIO	Hinweis
LED-Streifen	GPIO3	Auf ESP8266 entspricht dies RX und nutzt die alte, feste Standard-Ansteuerung.
Power-Taste	GPIO2	Taster gegen GND; wird per internem Pull-up gelesen.
Mode-Taste	GPIO13	Kurzer Druck: nächster Modus; langer Druck: nächste Transition.
Speed-Taste	GPIO14	Kurzer Druck: Helligkeit; langer Druck: Farbton.

Tabelle 3.1: Standardbelegung der konfigurierbaren Hardware-Pins

Die Taster werden mit `INPUT_PULLUP` betrieben. Ein Taster verbindet den GPIO daher beim Drücken mit GND. Für alle vier Hardware-Pins gilt: Jeder GPIO darf nur einmal verwendet werden. Die Firmware prüft doppelte Pins und ungültige Werte und stellt bei ungültigen EEPROM-Werten wieder die Standard-Pins her.

3.3.2 Geeignete Pins je Controller

Die folgenden Empfehlungen beziehen sich auf die in diesem Projekt verwendeten Controller-Varianten. Zusätzlich muss geprüft werden, welche Pins auf dem konkreten Entwicklungsboard wirklich herausgeführt sind. Einige Hersteller drucken Board-Bezeichnungen wie

«D1» oder «RX» auf; in der Web-Konfiguration wird trotzdem die GPIO-Nummer eingetragen.

ESP8266 / ESP8285

Für ESP8266-Boards akzeptiert die Firmware GPIO0 bis GPIO16. Empfehlenswert sind:

- LED-Streifen: Standard GPIO3/RX. Alternativ GPIO4, GPIO5, GPIO12, GPIO13 oder GPIO14.
- Hardware-Taster: GPIO4, GPIO5, GPIO12, GPIO13 oder GPIO14. GPIO2 ist als Standard für die Power-Taste nutzbar, darf beim Booten aber nicht auf LOW gezogen werden.
- Nicht verwenden: GPIO6 bis GPIO11, da diese üblicherweise mit dem Flash-Speicher verbunden sind.
- Mit Vorsicht verwenden: GPIO0, GPIO2 und GPIO15 sind Boot-Strapping-Pins. Falsche Pegel beim Einschalten können den Bootvorgang verhindern. GPIO16 hat keinen normalen internen Pull-up und ist deshalb für die Taster-Schaltung nicht empfohlen.

ESP32

Für ESP32-Boards akzeptiert die Firmware GPIO0 bis GPIO39. Empfehlenswert sind:

- LED-Streifen: GPIO4, GPIO13 bis GPIO19, GPIO21 bis GPIO23, GPIO25 bis GPIO27, GPIO32 oder GPIO33. GPIO3 bleibt als Standard aus Kompatibilitätsgründen möglich, liegt aber oft auf RX0.
- Hardware-Taster: GPIO4, GPIO13 bis GPIO19, GPIO21 bis GPIO23, GPIO25 bis GPIO27, GPIO32 oder GPIO33.
- Nicht verwenden: GPIO6 bis GPIO11, da diese üblicherweise mit dem SPI-Flash verbunden sind.
- Nicht für den LED-Streifen verwenden: GPIO34 bis GPIO39, da diese nur Eingang sind. Für Taster sind sie nur mit externem Pull-up geeignet, da sie keine internen Pull-up-/Pull-down-Widerstände besitzen.

- Mit Vorsicht verwenden: GPIO0, GPIO2, GPIO5, GPIO12 und GPIO15 sind Boot-Strapping-Pins. GPIO1 und GPIO3 sind die Standard-UART-Pins und können die serielle Ausgabe stören.

ESP32-C3

Der ESP32-C3 besitzt GPIO0 bis GPIO21. Auch wenn die Weboberfläche höhere Zahlen nicht verhindert, sollten nur tatsächlich vorhandene Pins des Boards verwendet werden. Empfehlenswert sind:

- LED-Streifen und Hardware-Taster: GPIO0, GPIO1, GPIO3, GPIO4, GPIO5, GPIO6, GPIO7, GPIO10, GPIO11, GPIO20 oder GPIO21, sofern diese Pins auf dem verwendeten Board herausgeführt sind.
- Nicht verwenden: GPIO12 bis GPIO17, da diese üblicherweise mit SPI-Flash oder PSRAM verbunden sind.
- Mit Vorsicht verwenden: GPIO2, GPIO8 und GPIO9 sind Boot-Strapping-Pins. GPIO18 und GPIO19 sind standardmäßig für USB-JTAG vorgesehen.
- Hinweis zu den Standard-Pins: GPIO13 und GPIO14 sind auf ESP32-C3-Boards häufig nicht als freie GPIOs nutzbar. Für ESP32-C3 sollten die Button-Pins daher vor dem Verdrahten auf geeignete freie GPIOs geändert werden.

Die Einschränkungen orientieren sich an den GPIO-Hinweisen der Espressif-Dokumentation für ESP32 und ESP32-C3 sowie den bekannten ESP8266-Boot- und Flash-Pins.

3.3.3 Bedienung der Hardware-Taster

- Power-Taste: schaltet die LEDs aus und bei erneutem Druck wieder mit den vorherigen Farben ein.
- Mode-Taste: kurzer Druck wechselt zum nächsten Anzeigemodus, langer Druck zur nächsten Transition.
- Speed-Taste: kurzer Druck erhöht die Helligkeit, langer Druck wechselt den Farbton.

4 Funktionen der Uhrensoftware auf dem ESP

4.1 Verbinden der Uhr mit dem eigenen WLAN

Nach dem erfolgreichen Verbinden kann die Uhr nach Eingabe der angezeigten IP-Adresse in einem Browser konfiguriert werden. Dabei kann die Uhr auch mit dem eigenen WLAN Netz verbunden werden.

4.2 Zeitserver (NTP Server)

Falls die Uhr an das eigene WLAN mit Internet angebunden wird, kann ein Zeitserver hinterlegt werden. Dieser sorgt dafür, dass die Uhrzeit zyklisch sekundengenau gestellt wird.

4.3 Automatische Helligkeitsregelung

Falls der optionale LDR an der Uhr angeschlossen wurde, ist es möglich, die Helligkeit automatisch, abhängig der Umgebungshelligkeit, regeln zu lassen. Dazu muss folgende Einstellung aktiviert werden. Durch das Feld Kalibrierung kann man die Grundhelligkeit an seine eigenen Wünsche anpassen.